



CROWD COUNTING PROJECT REPORT

ELIA ANTONIO MALVÈ

Table of contents

Problem	2
Dataset	2
Training dataset	2
Test dataset	3
Data labeling	4
Methods	4
Learning algorithm	4
CNN architecture.....	4
Valutation.....	4
Experiments.....	5
Choosing a model.....	5
Choosing hyperparameters	5
Demo.....	9
Code.....	10
Conclusion	10

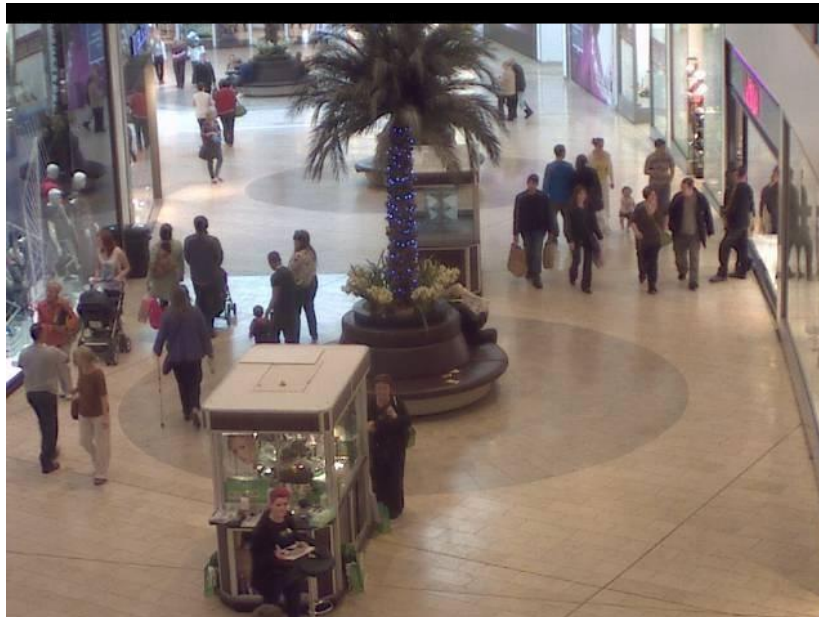
Problem

The project aims to develop a model for calculating the number of people in an image - a task known as crowd counting - based on a convolutional neural network (CNN). The algorithm is trained on a publicly available dataset containing crowd images, each annotated with the number of people present, and is subsequently tested on a custom-generated dataset. The objective is to estimate the number of individuals in realistic scenarios using regression. Potential applications for this software range from calculating and optimizing crowd numbers in shopping malls to maximize revenue, to monitoring enclosed areas for security purposes.

Dataset

Training dataset

The dataset used ([accessible here](#)), was built from public resources available online, in this case by Kaggle, a site that represents a reference in the field of data science. It contains 2000 RGB images at a resolution of 480x640 pixels, which show different frames of the same camera in a shopping center, as the pedestrian traffic present varies. Together with the images, the labels for each image are included in the dataset, showing the number of people present.

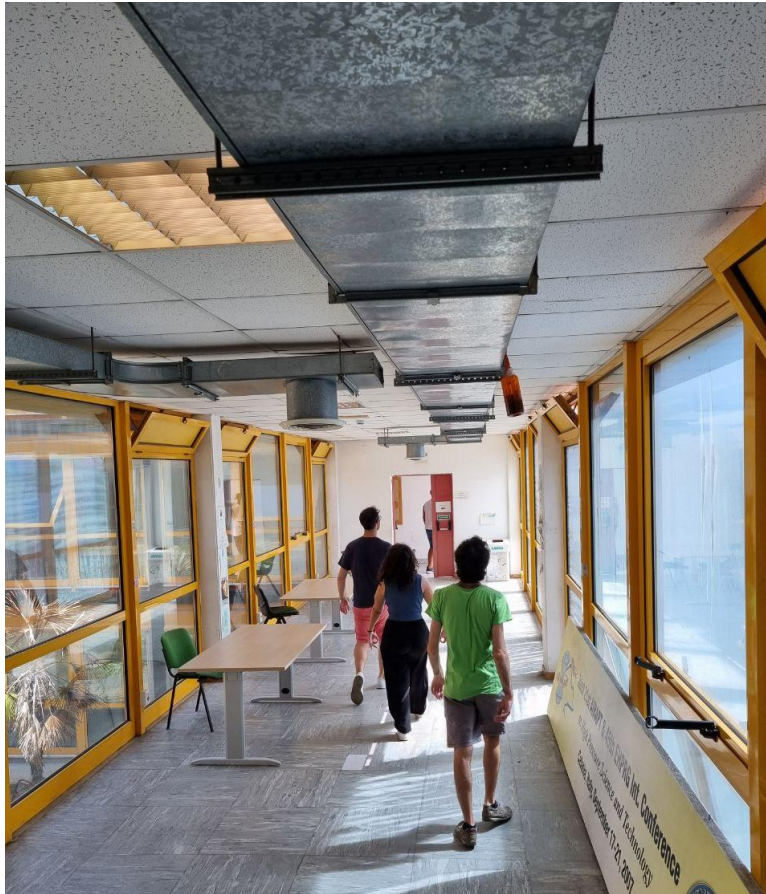


Example image of the train dataset.

Each image was pre-processed (resized to 224x224 pixels) and normalized according to the standard ImageNet parameters. Data augmentation techniques (rotations, flips, zooms, color jitter) were also applied to improve the generalization of the model.

Test dataset

The test dataset was built by selecting a set of independent images, not used in the training or validation phase, to have a realistic evaluation of the model's ability to generalize. Several images were joined, some of which can be freely downloaded from the internet, others instead produced punctually for the purpose. The label is inherent in the format of the name, such as "test_####_%%%%", with #### an incremental id that uniquely identifies the image, and %%%% the label, i.e. the number of people present. In total, 50 images were used per test, of which 30 were extracted from the internet and 20 were produced by the author specifically for the purpose (with the help of several volunteers). All were chosen in different lighting conditions and framing, to evaluate the generalization capacity of the model. Also, on these we have resized to 224x224 pixels, necessary for the input to the CNN used, and have normalized with the standard parameters of ImageNet, but no data augmentation has been carried out.



Sample image of the test dataset.

Data labeling

The labels were provided in the form of counts (number of individuals per image). The model was trained to predict a numerical value, trying to optimize the distance between the prediction and the actual count. The labeling of the test data was done manually.

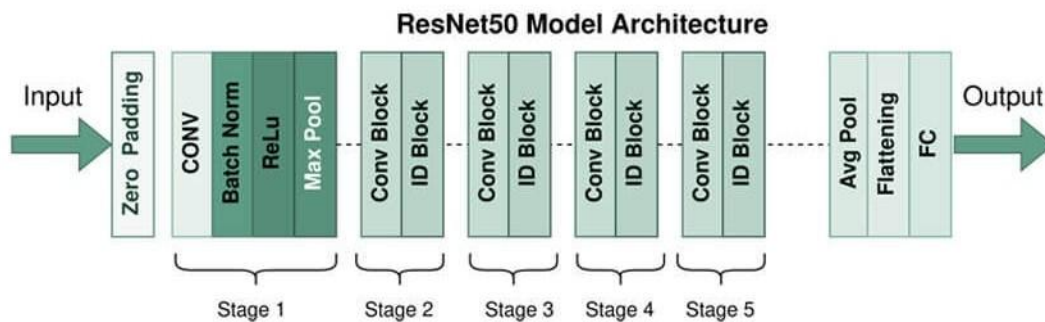
Methods

Learning algorithm

The operation of the CNN is based on several phases. The first step is the **preprocessing** of the images, where they were scaled to 224x224 pixels, and normalized with the ImageNet parameters, so that they could be given as input to a CNN, specifically ResNet50. In the case of the train and validation dataset, data augmentation was also carried out, as described above. A transfer learning strategy was adopted, using the pre-trained ResNet50 as a backbone. and dividing the entire dataset into training set (80%) and validation set (20%). **The optimization** was performed by Stochastic Gradient Descent (SGD) with momentum and regularization parameter. The progress of the training is also visible via tensorboard in real time.

CNN architecture

The network chosen is ResNet50, a CNN characterized by residual connections (the output of one layer is brought to the next layer) that reduce the problem of vanishing gradient and exploding gradient. At the exit of the network, a fully connected layer was adapted to predict a single value (the estimated count of people). A dropout (random shutdown of some neurons in the network, in the training phase) of 55% was applied to reduce overfitting.



ResNet50 architecture schema.

Valutation

During the training, the metrics Mean Absolute Error (MAE), Mean Squared Error (MSE) and Root Mean Squared Error (RMSE) were monitored. In particular, the MAE was considered the most reliable indicator, as it makes physical sense, as it is an error on the

count. The final model and its hyperparameters were selected based on the performance on the validation set.

Experiments

Choosing a model

Several pre-trained models (SqueezeNet, ResNet18, ResNet34, ResNet50) were tested. The first attempt was made with the SqueezeNet architecture, as it was particularly light and already seen during the educational workshops. However, the results were not very satisfactory, so it was decided to turn towards a more complex CNN, such as ResNet. After a few attempts in this direction, ResNet50 guaranteed a good compromise between accuracy and computational cost, also considering the complexity of the subsequent ones (ResNet101, ResNet 152) and therefore it was used in all subsequent phases.

Choosing hyperparameters

Several tests were carried out, mainly varying the learning rate, momentum, regularization parameter and dropout. In particular, the following attempts were made to identify the best conditions of parameters:

ATTEMPT	LEARNING RATE	MOMENTUM	REGULARIZATION PARAMETER	DROPOUT
1	0.001	0.95	0.0001	0.60
2	0.001	0.94	0.0001	0.57
3	0.001	0.93	0.0001	0.58
4	0.0001	0.93	0.0001	0.40
5	0.0005	0.93	0.0001	0.45
6	0.001	0.93	0.0001	0.60
7	0.001	0.92	0.0001	0.55

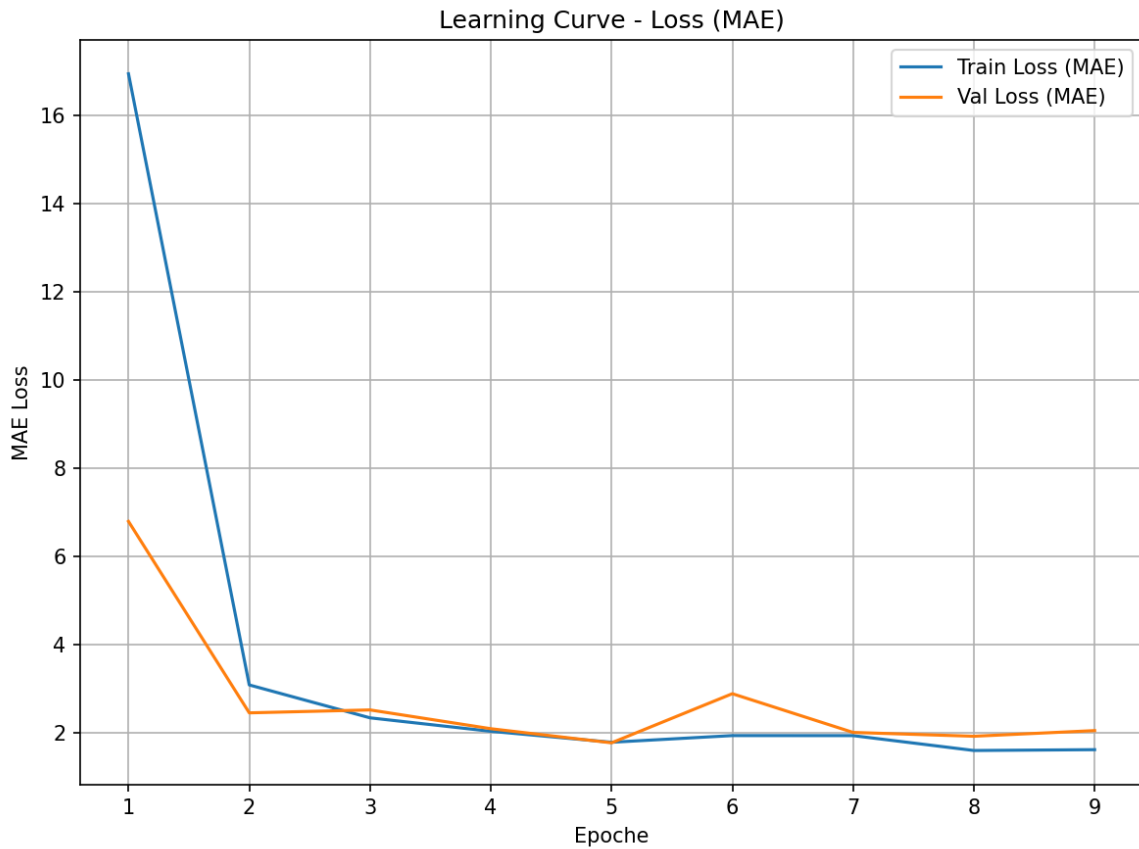
Having obtained the best parameters in configuration 7, they were fixed, proceeding to vary only one of them at a time for a fine calibration. In particular, the momentum in the range [0.90; 0.99] and the dropout in the range [0.50; 0.60] were varied, the latter in order to prevent overfitting as best as possible. Dynamic reduction of the learning rate was also implemented, to avoid local minimums of the loss function if there is no appreciable gain for a certain number of epochs.

The final hyperparameters were:

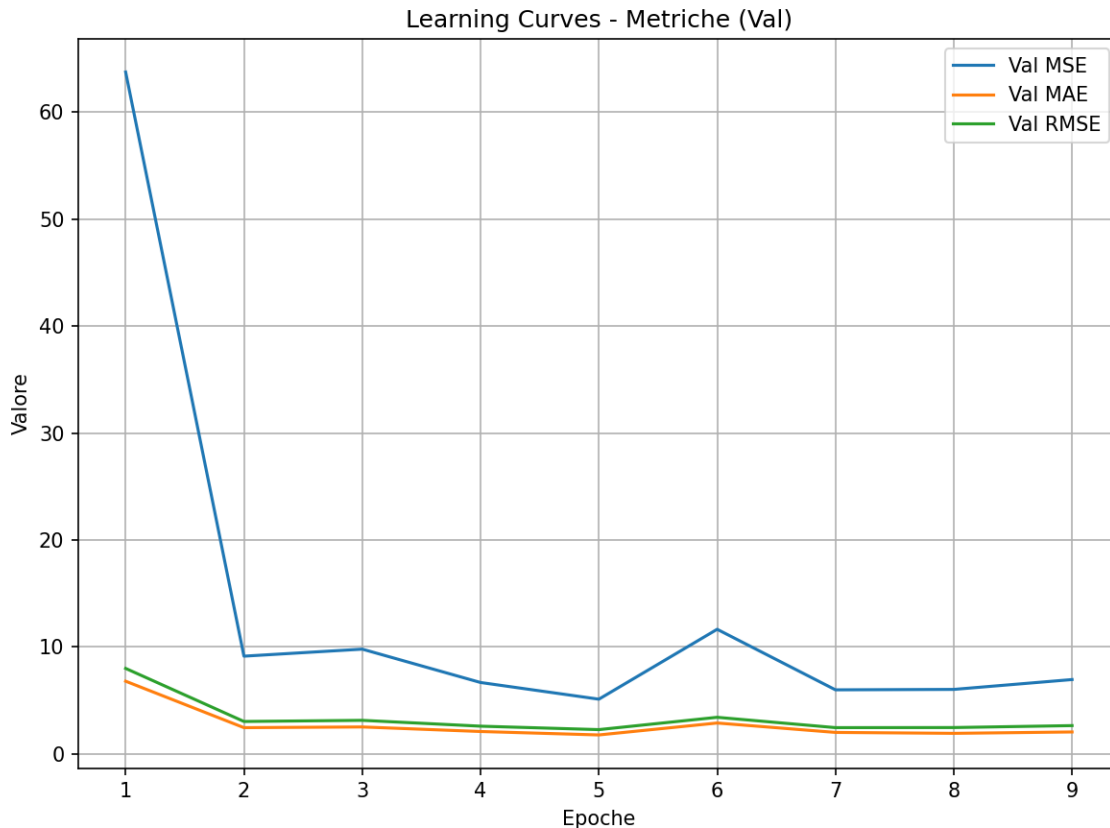
- Batch size: 32 (unchanged in all runs, number of samples per SGD)
- Learning rate: 0.001

- Momentum: 0.92
- Weight decay: 0.0001
- Dropout: 0.55
- Patience: 4 epochs (epochs without improvement sufficient to stop the algorithm)

The training stopped after 9 epochs due to early stopping.



Loss curves per train (y) and test set as the epochs (x) change. In the first two epochs, the validation loss is lower than the train loss due to the dropout, which is present only in the train phase. The trend is very similar to the expected one.



Trend of metrics with changing eras.

Results on the validation set:

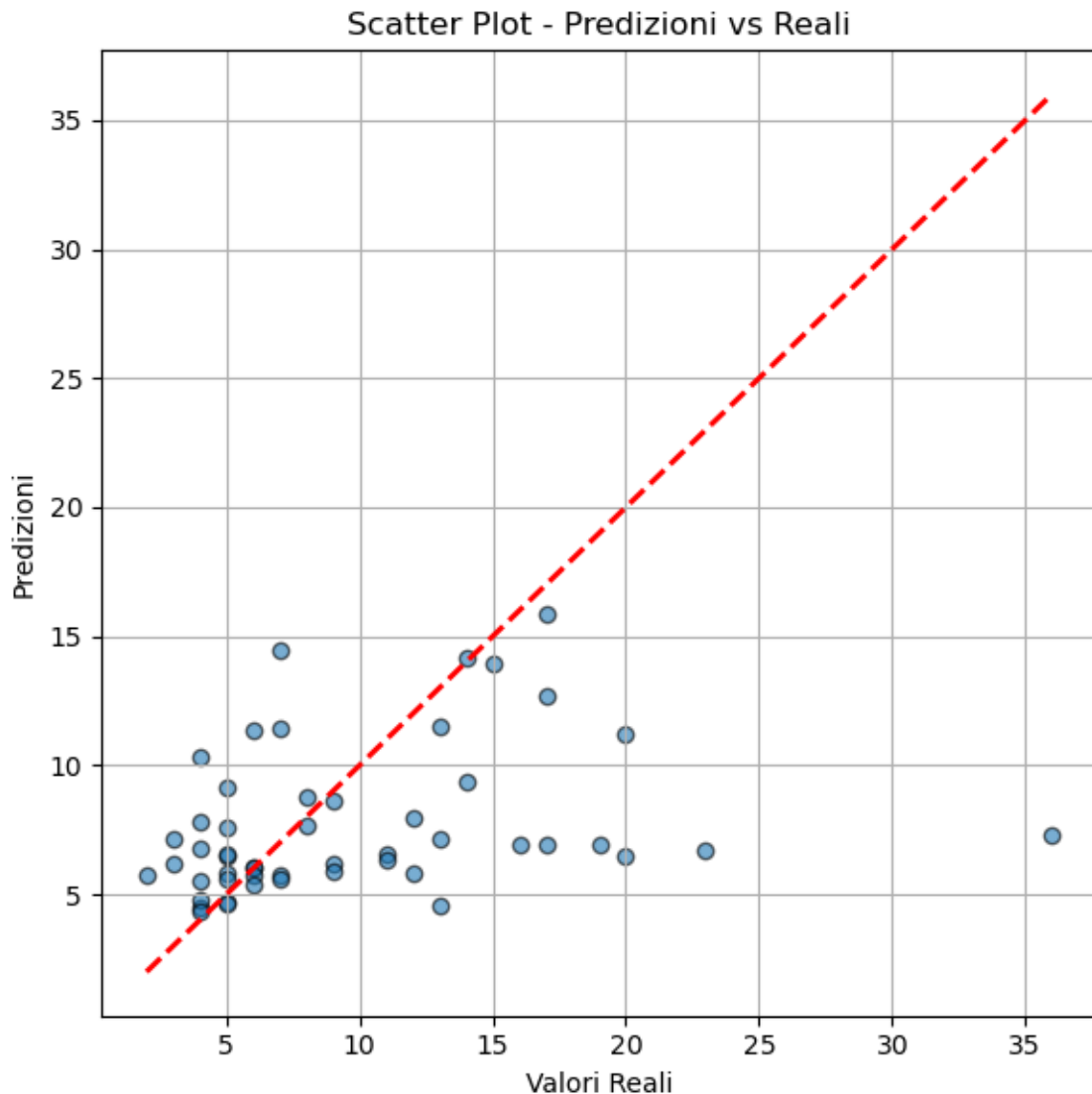
- MSE: 5.1072
- MAE: 1.7629
- RMSE: 2.2599

Using the MAE as a reference, the model misses the count, on average, of 1.76 people on the validation set, an acceptable result considering the size of the dataset and the configuration of the photos in terms of framing.

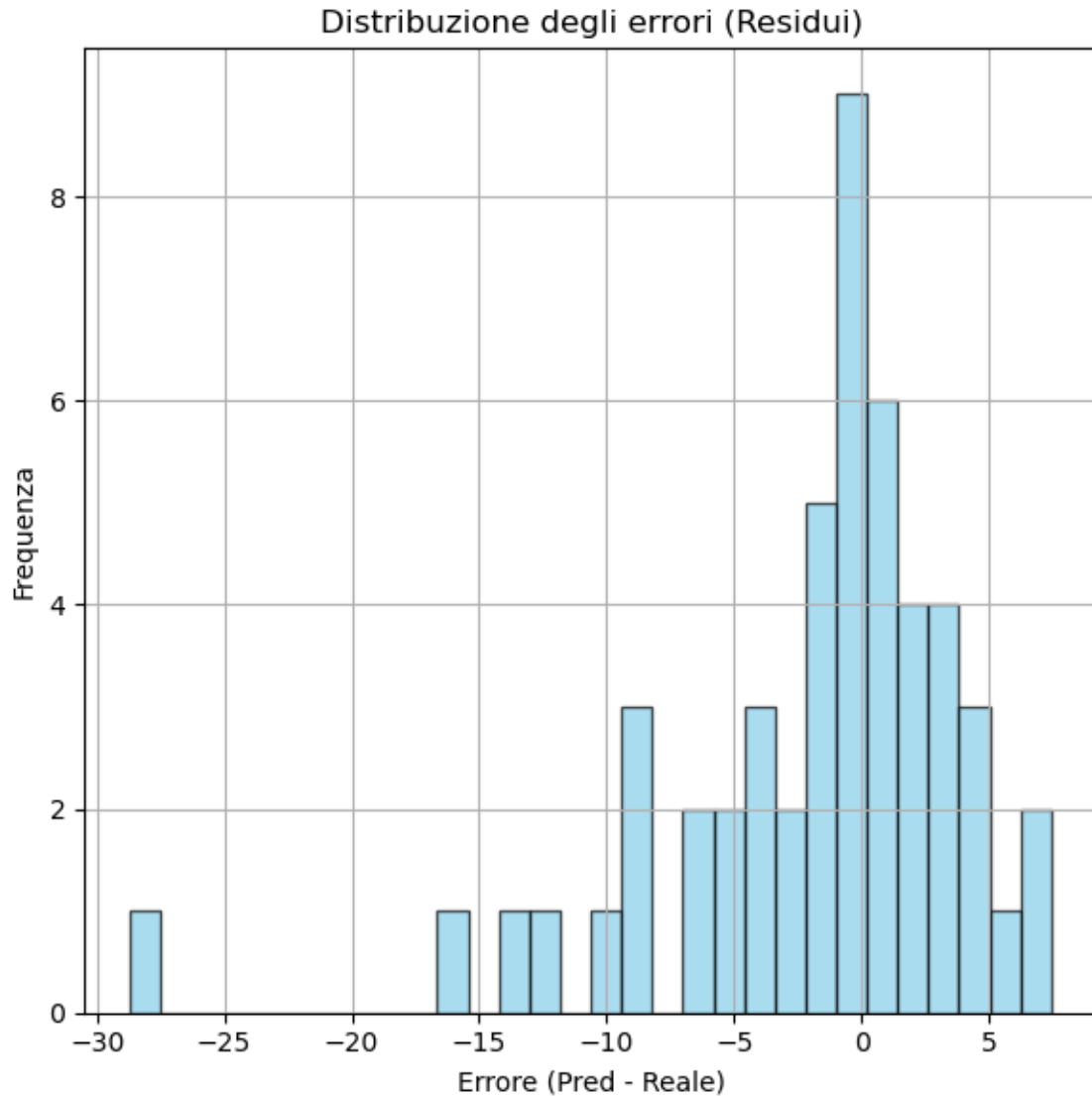
Results on the test set:

- MSE: 42.9007
- MAE: 4.1608
- RMSE: 6.5499
- MAE confidence interval (95%): [2.9183, 5.5297] (This last parameter was inserted because, due to the small size of the test set, the results of the metrics obtained can fluctuate even with the addition of a few elements. Therefore, this represents the maximum and minimum MAE that can be obtained, a form of error on the metric).

The metrics obtained on the test dataset differ significantly from the values obtained in the training. Several factors contribute to this phenomenon, first of all the rather limited size of the test dataset, which also has different shots, light conditions and environment than the dataset used for training. By virtue of the occurrence of these conditions, the worsening of the metric values is not excessively high, also taking into consideration that it is not an excessively complex model.



Scatter plot showing the true count values (x) with their predictions (y). The model seems to make predictions better with a low number of people in the image. The points on the bisector are those that have the same number of predictions and true values, so the closer a point is to the line in question, the better the prediction will have been.



Frequency (y) and distribution of prediction errors (x) calculated as (predicted value - true value). The peak on value 0 indicates that the model predicts correctly in most cases, despite clearly evincible errors, mostly at overcounting.

Demo

A Python demo has been implemented using the Streamlit library, which allows you to create interactive web interfaces, where an area for uploading images has been inserted and immediately below the prediction of the number of individuals. In particular, the interface allows you to compare the estimated value with the real one, since there are two areas on the same page, one for the prediction and another for the ground truth label, extracted from the filename if it is in the expected format, discussed previously in the dataset paragraph. If the filename is not in the expected format, an error message is displayed, but the predictions are still shown.

Code

The project is organized as:

- `code\`: the main folder, inside you have the three source files `contapersone_resnet50.py`, which is the code for the train of the model, `valutazione_test.py`, which is the code to test the model on the test set, finally `demo.py`, which is the script for the demo. If the train phase is repeated, the model file is overwritten;

The instructions for the correct operation of the codes are present, in the form of comments, in the headers of the codes themselves. The latter are also commented punctually in each line deemed necessary, to make clear the function performed.

Anyway, the command to boot from terminal is `python (source file name).py`.

Conclusion

The developed model can estimate the number of people present in crowd images obtaining an average MAE of about 4.16 on the test set. Despite the good generalization, difficulties persist in cases where the context, or light conditions, of the test images are very different from that of the training set. Future improvements could include the use of newer and more powerful architecture (such as EfficientNet or Vision Transformers), and/or a greater amount of training data, obtained in more varied situations.

This project was very useful to consolidate what I learned from the study of theory and better understand how a CNN works and its implementation. In addition, it allowed me to deal with the typical problems related to the training of a convolutional neural network, such as overfitting, the choice of hyperparameters and the dataset, without forgetting what I learned in general about the Python language.